Part 1: Show the details of the MapReduce Method

(1.a) Map Phase Inputs

The inputs that the MapReduce method will receive will be the rows in the two databases. We're going to lump all those tuples into one dataset with labels.

- Employees: (Adina, Computer Science)
- Employees: (Alex, Mathematics)
- Employees: (David, Computer Science)
- Employees: (Felicia, Mathematics)

- Departments: (Computer Science, Turing)
- Departments: (Mathematics, Euler)

(1.b) Map Phase Outputs

Then, the map transforms these into a key-value pair where the key will be the join attribute, "Department"

- Key: "Computer Science" → Value: "Employee:Adina"
- Key: "Mathematics" → Value: "Employee:Alex"
- Key: "Computer Science" → Value: "Employee:David"
- Key: "Mathematics" → Value: "Employee:Felicia"

- Key: "Computer Science" → Value: "Head:Turing"
- Key: "Mathematics" → Value: "Head:Euler"

(1.c) Shuffle Phase Outputs

Then, the shuffle phase groups all values by the same key (Department):

Grouped by Department:

- Key: "Computer Science" → Values: ["Employee:Adina", "Employee:David", "Head:Turing"]
- Key: "Mathematics" → Values: ["Employee:Alex", "Employee:Felicia", "Head:Euler"]

(1.d) Reduce Phase Outputs

Finally. the reduce phase joins by combining employees with their department heads:

Final Join Results:

- (Adina, Computer Science, Turing)
- (David, Computer Science, Turing)
- (Alex, Mathematics, Euler)
- (Felicia, Mathematics, Euler)

Part 2: MapReduce Pseudocode

(2.a) map() pseudocode

```
map(String input_key, String input_value):
   // input_key: row number from the databases
   // input_value: row values from the corresponding row

   // Parse the input record
   fields = parse(input_value)

   // Check if this row is from the Employees table (has 2 fields: name and department)
   if (isEmployeeRecord(fields)):
      // Employee Format: "Employee,Department"
      employee_name = fields[0]
      department = fields[1]

      // Use department as key so all employees and heads in same dept get grouped together
      EmitIntermediate(department, "Employee:" + employee_name)

   // Check if this row is from the Departments table (has 2 fields: department and head)
   else if (isDepartmentRecord(fields)):
      // Department Format: "Department,Head"
      department = fields[0]
      head = fields[1]

      // Emit with department as key
      EmitIntermediate(department, "Head:" + head)
```

(2.b) Explanation of map() function

1. **Input Processing**: The function takes in each record from either the Employees or Departments table as input
2. **Record Identification**: It parses the input to determine which table the record belongs to
3. **Key Selection**: Uses the join attribute, "Department" as the output key
4. **Value Tagging**: Tags each value with its source table ("Employee:" or "Head:") so the reduce function can distinguish between them
5. **Emission**: Outputs the key-value pair for the shuffle phase

(2.c) reduce() pseudocode

```
reduce(String output_key, Iterator intermediate_values):
  // output_key: department name (the join attribute)
  // intermediate_values: all employees and heads for this department

  // Create separate lists for employees and department heads in this department
  employees = []
  heads = []

  // Sort each value into the appropriate list based on its tag
  for each value in intermediate_values:
    if (value.startsWith("Employee:")):
      // Remove "Employee:" prefix to get just the name (e.g., "Adina")
      employee_name = value.substring(length("Employee:"))
      employees.append(employee_name)

    else if (value.startsWith("Head:")):
      // Get department head and add to list
      head_name = value.substring(length("Head:"))
      heads.append(head_name)

  // Do the natural join
  for each employee in employees:
    for each head in heads:
      // Output format: "Employee Name, Department, Head"
      // Example output: "Adina, Computer Science, Turing"
      EmitFinal(employee + ", " + output_key + ", " + head)
```

(2.d) Explanation of reduce() function

1. **Input Grouping**: Receives all data for one specific department (the join key)
2. **Value Separation**: Splits the grouped values into two
    - Employees List
    - Department Heads list
3. **Join Operation**: Does the final join:
    - Taking each employee from the employees list
    - Pairing them with each head from the heads list
4. **Output Generation**: Outputs the joined records:
    - "Adina, Computer Science, Turing" (Adina works in CS, which is headed by Turing)